Praktikum JavaScript & TypeScript

Modul 4 - String dan Object

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, peserta mampu:

- Mengolah teks dengan method string: mengubah huruf, membuang spasi, memotong, mencari, dan mengganti
- Memecah string menjadi array dan menggabungkannya kembali (`split` dan `join`)
- Membuat, membaca, dan mengubah object, termasuk dengan key dari variabel
- Memakai object sebagai penghitung per kelompok
- Mengambil isi object dan array dengan *destructuring*, menyalin dan menggabungkan dengan *spread*
- Menjelaskan beda salinan dangkal dan salinan dalam
- Mengubah object menjadi teks JSON dan sebaliknya

2. Dasar Teori

2.1. String Itu Apa

String adalah teks: deretan karakter di antara tanda kutip. Kutip satu `'...'`, kutip dua `"..."`, dan backtick `...` sama-sama membuat string. Hanya backtick (*template literal*) yang bisa menyisipkan nilai dengan `${...}` dan menulis beberapa baris.

```
const a = 'Ani';
const b = "Ani";
const umur = 20;
const c = `${b} berumur ${umur} tahun`;   // "Ani berumur 20 tahun"
const d = `baris 1
baris 2`;                                 // string dua baris, pemisahnya karakter "\n"
```

Listing 1: tiga cara menulis string

2.1.1. String mirip array, tetapi bukan array

String punya `length` dan setiap karakternya bisa dibaca lewat index, persis seperti array. String juga bisa diulang dengan `for...of`.

```
const kata = "Halo";
console.log(kata.length);   // 4
console.log(kata[0]);       // "H"   (index mulai dari 0)
console.log(kata[3]);       // "o"   (index terakhir = length - 1)
console.log(kata[10]);      // undefined (di luar batas, tidak error)
for (const huruf of kata) {
  console.log(huruf);       // H, a, l, o (satu per baris)
}
```

Listing 2: membaca string seperti array

Tiga perbedaan dengan array:

1. **String tidak punya method array** seperti `map`, `filter`, dan `reduce`. Untuk memakainya, ubah dulu menjadi array dengan `kata.split("")` atau `[...kata]`. Hasilnya `["H", "a", "l", "o"]`.
2. **String tidak bisa diubah** (*immutable*). Tidak ada cara mengganti satu huruf di dalam string yang sudah ada. Semua method string **mengembalikan string baru** dan membiarkan string asli apa adanya.
3. Karena itu, hasil method string harus **ditangkap** ke variabel atau langsung dipakai. Memanggil method tanpa menangkap hasilnya tidak mengubah apa pun.

```
const nama = "budi";
nama.toUpperCase();            // hasil "BUDI" dibuang, tidak ditangkap
console.log(nama);             // "budi"  (tetap)
const besar = nama.toUpperCase();
console.log(besar);            // "BUDI"
```

Listing 3: string tidak berubah oleh method-nya

> **⚠ Perhatian**
>
> Menulis ke index string, misalnya `nama[0] = "B"`, tidak pernah berhasil. Di file kita (ES module, mode *strict*), baris itu melempar `TypeError: Cannot assign to read only property '0' of string 'budi'`. Di mode lama (bukan strict), baris itu diam-diam diabaikan, jadi lebih susah dilacak. Untuk "mengganti" huruf, buat string baru dari potongan-potongannya (lihat `slice` di bawah).

2.2. Memilih Method String dari Pertanyaannya

Seperti Modul 3, mulai dari pertanyaannya. Kolom SQL hanya pembanding.

Method	Pertanyaan yang dijawab	Hasil	Mirip SQL
`length`	Berapa karakternya?	number	`LENGTH`
`toUpperCase`, `toLowerCase`	Ubah semua huruf jadi kapital atau kecil	string baru	`UPPER`, `LOWER`
`trim`	Buang spasi di awal dan akhir	string baru	`TRIM`
`slice`	Ambil sebagian karakter	string baru	`SUBSTRING`
`includes`	Apakah potongan ini ada di dalamnya?	boolean	`LIKE '%x%'`
`startsWith`, `endsWith`	Apakah diawali atau diakhiri potongan ini?	boolean	`LIKE 'x%'`
`indexOf`	Di index berapa potongan ini mulai?	number, atau `-1`	`POSITION`
`replace`, `replaceAll`	Ganti potongan teks	string baru	`REPLACE`
`padStart`, `padEnd`	Isi depan atau belakang sampai panjang tertentu	string baru	`LPAD`, `RPAD`
`repeat`	Ulang string sebanyak n kali	string baru	`REPEAT`
`split`	Pecah string menjadi array	**array** string	`STRING_SPLIT`

`join` adalah method **array**, kebalikan `split`: array digabung menjadi string. Tidak ada method yang mengubah string asli.

2.2.1. toUpperCase, toLowerCase, trim

Untuk apa: merapikan input pengguna sebelum dibandingkan atau ditampilkan. Pengguna sering mengetik `" ANI "`, `"ani"`, atau `"Ani"` untuk nama yang sama.

Cara kerja: `toUpperCase()` dan `toLowerCase()` mengubah semua huruf. `trim()` membuang spasi (juga tab dan baris baru) di **awal dan akhir** saja. Spasi di tengah tidak disentuh. Ketiganya bisa di-*chain*, karena hasilnya string.

```
const input = "  ANI  ";
console.log(input.trim());                  // "ANI"
console.log(input.trim().toLowerCase());    // "ani"
console.log(input.trim().toLowerCase() === "ani");   // true
console.log("  dua   spasi  ".trim());      // "dua   spasi"  (spasi tengah tetap)
```

Listing 4: merapikan sebelum membandingkan

Kesalahan umum: lupa tanda kurung. `nama.toUpperCase` tanpa `()` adalah fungsinya, bukan hasilnya. Di template literal tercetak `function toUpperCase() { [native code] }`.

2.2.2. slice: ambil sebagian

Untuk apa: mengambil potongan string. Contohnya huruf pertama, 3 huruf awal, atau 4 karakter terakhir.

Cara kerja: `slice(awal, akhir)` mengambil dari index `awal` sampai **sebelum** index `akhir`. Jika `akhir` tidak ditulis, potongan berjalan sampai ujung. Index negatif dihitung dari belakang: `-1` adalah karakter terakhir. Sama seperti `slice` pada array di Modul 3.

```
const s = "Santoso";
console.log(s.slice(0, 3));    // "San"   (index 0, 1, 2)
console.log(s.slice(1));       // "antoso" (dari index 1 sampai ujung)
console.log(s.slice(-4));      // "toso"  (4 karakter terakhir)
console.log("Ab".slice(0, 5)); // "Ab"    (melewati batas tidak error)
```

Listing 5: slice pada string

Karena string tidak bisa diubah, “mengganti huruf pertama” berarti menyusun string baru: huruf pertama yang sudah diubah, disambung sisanya.

```
const kata = "jAKARTA";
const rapi = kata[0].toUpperCase() + kata.slice(1).toLowerCase();
console.log(rapi);   // "Jakarta"
```

Listing 6: huruf pertama kapital

Baca dari kiri: `kata[0]` adalah `"j"`, lalu menjadi `"J"`. `kata.slice(1)` adalah `"AKARTA"`, lalu menjadi `"akarta"`. Keduanya disambung dengan `+`.

Kesalahan umum: memanggil `kata[0].toUpperCase()` pada string kosong. `""[0]` bernilai `undefined`, sehingga muncul `TypeError: Cannot read properties of undefined (reading 'toUpperCase')`. Pastikan string tidak kosong sebelum mengambil huruf pertamanya.

2.2.3. includes, startsWith, endsWith, indexOf: mencari

Untuk apa: memeriksa isi teks. Contohnya apakah email memuat `@`, apakah kode diawali `"INV"`, atau di mana tanda `-` berada.

Cara kerja: `includes`, `startsWith`, dan `endsWith` menjawab ya atau tidak. `indexOf` mengembalikan index awal potongan, atau `-1` jika tidak ada. Semuanya membedakan huruf besar dan kecil.

```
const email = "ani@kampus.ac.id";
console.log(email.includes("@"));        // true
console.log(email.endsWith(".ac.id"));   // true
console.log(email.startsWith("Ani"));    // false (huruf A besar berbeda)
console.log(email.indexOf("@"));         // 3
console.log(email.indexOf("#"));         // -1
```

Listing 7: mencari di dalam string

Kesalahan umum: memakai `indexOf(...)` langsung sebagai kondisi. `if (email.indexOf("a"))` bernilai `false` ketika `"a"` ada di index 0, karena 0 itu falsy. Untuk pertanyaan ya atau tidak, pakai `includes`.

2.2.4. replace dan replaceAll: mengganti

Untuk apa: mengganti potongan teks. Contohnya menghapus tanda hubung dari nomor atau mengganti pemisah.

Cara kerja: `replace(cari, ganti)` hanya mengganti kemunculan **pertama**. `replaceAll(cari, ganti)` mengganti semuanya. Keduanya mengembalikan string baru.

```
const nomor = "0812-3456-7890";
console.log(nomor.replace("-", ""));      // "08123456-7890" (hanya yang pertama)
console.log(nomor.replaceAll("-", ""));   // "081234567890"
```

Listing 8: replace dan replaceAll

2.2.5. padStart dan repeat: mengisi dan mengulang

Untuk apa: membuat teks dengan panjang tetap. Contohnya nomor faktur `"007"` atau sensor `"****"`.

Cara kerja: `padStart(panjang, pengisi)` menambahkan `pengisi` di depan sampai panjang string mencapai `panjang`. Jika string sudah sepanjang itu atau lebih, string dikembalikan apa adanya. `repeat(n)` mengulang string `n` kali.

```
console.log("7".padStart(3, "0"));     // "007"
console.log("42".padStart(3, "0"));    // "042"
console.log("1234".padStart(3, "0"));  // "1234" (sudah lebih panjang)
console.log("*".repeat(5));            // "*****"
console.log("ab".repeat(0));           // ""     (string kosong)
```

Listing 9: padStart dan repeat

> **⚠ Perhatian**
>
> `padStart` adalah method **string**, bukan number. `const n = 7; n.padStart(3, "0")` melempar `TypeError: n.padStart is not a function`. Ubah dulu angkanya menjadi string: `String(n).padStart(3, "0")`.

2.2.6. Number dan string

Angka dari teks (misalnya hasil `split`) masih bertipe string. Ingat Modul 1: `+` pada string menyambung, bukan menjumlah.

Kode	Hasil	Keterangan
`"5" + 1`	`"51"`	`+` dengan string menyambung
`"5" - 1`	`4`	`-` memaksa menjadi number
`Number("42")`	`42`	string ke number
`Number(" 42 ")`	`42`	spasi di tepi diabaikan
`Number("abc")`	`NaN`	bukan angka
`Number("")`	`0`	string kosong menjadi 0, hati-hati
`String(7)`	`"7"`	number ke string

2.3. split dan join: Jembatan String dan Array

Untuk apa: mengolah teks per bagian. Teks dipecah menjadi array dengan `split`, array diolah dengan method Modul 3 (`map`, `filter`), lalu digabung kembali dengan `join`.

Cara kerja: `split(pemisah)` memotong string di setiap `pemisah` dan mengembalikan **array string**. Pemisahnya sendiri dibuang. `join(pemisah)` menyisipkan `pemisah` di antara elemen array dan mengembalikan **string**.

```
console.log("apel,jeruk,mangga".split(","));   // ["apel", "jeruk", "mangga"]
console.log("a,b,,c".split(","));              // ["a", "b", "", "c"]  (ada string kosong)
console.log("Halo".split(""));                 // ["H", "a", "l", "o"]
console.log(["2026", "11", "04"].join("-"));   // "2026-11-04"
console.log("satu".split(","));                // ["satu"] (pemisah tidak ada: 1 elemen)
```

Listing 10: split dan join

> **⚠ Perhatian**
>
> Dua pemisah yang berdampingan menghasilkan **string kosong** `""` di antaranya. Contohnya `"a,b,,c"` di atas, atau `"dua  spasi".split(" ")` yang menghasilkan `["dua", "", "spasi"]`. Buang elemen kosong dengan `filter((x) => x !== "")`.

2.3.1. Pola: pecah, olah per bagian, gabung

Banyak tugas string berbentuk sama: pecah menjadi array, ubah atau saring setiap bagian, lalu gabung lagi. Contohnya mengubah judul artikel menjadi *slug* untuk URL: `"  Belajar   JS Dasar "` menjadi `"belajar-js-dasar"`.

```
function buatSlug(judul) {
  return judul
    .trim()                      // "Belajar   JS Dasar"
    .toLowerCase()               // "belajar   js dasar"
    .split(" ")                  // ["belajar", "", "", "js", "dasar"]
    .filter((k) => k !== "")     // ["belajar", "js", "dasar"]
    .join("-");                  // "belajar-js-dasar"
}
console.log(buatSlug("  Belajar   JS Dasar "));   // "belajar-js-dasar"
```

Listing 11: membuat slug

Bentuk data di setiap langkah:

Langkah	Nilai setelah langkah	Bentuk
awal	`"  Belajar   JS Dasar "`	string
`trim()`	`"Belajar   JS Dasar"`	string
`toLowerCase()`	`"belajar   js dasar"`	string
`split(" ")`	`["belajar", "", "", "js", "dasar"]`	array string
`filter(...)`	`["belajar", "js", "dasar"]`	array string
`join("-")`	`"belajar-js-dasar"`	string

Di tengah rantai, bentuknya array, jadi method array boleh dipakai. Di ujung, `join` mengembalikannya menjadi string. Jika setiap kata perlu diubah (bukan hanya disaring), tambahkan `map` sebelum `join`. Callback `map` menerima **satu kata** dan mengembalikan kata yang sudah diubah.

```
const kode = "a1 b2 c3".split(" ").map((k) => k.toUpperCase()).join("/");
console.log(kode);   // "A1/B2/C3"
```

Listing 12: map di tengah pola

Kesalahan umum: memanggil `join` pada string, misalnya `"abc".join("-")`, menghasilkan `TypeError: "abc".join is not a function`. `join` hanya ada di array. Sebaliknya, `split` hanya ada di string.

2.4. Object

2.4.1. Membuat dan membaca object

Object menyimpan beberapa nilai dengan nama (*key*). Di Modul 3, object dipakai sebagai satu baris tabel. Di sini kita melihatnya lebih dalam.

```
const buku = {
  judul: "Laskar Pelangi",
  tahun: 2005,
  penulis: { nama: "Andrea Hirata", kota: "Belitung" },   // object di dalam object
  tag: ["novel", "pendidikan"],                           // array di dalam object
};
console.log(buku.judul);           // "Laskar Pelangi"
console.log(buku.penulis.kota);    // "Belitung"
console.log(buku.tag[1]);          // "pendidikan"
console.log(buku.harga);           // undefined (key tidak ada, tidak error)
```

Listing 13: object literal

2.4.2. Dot notation dan bracket notation

Ada dua cara menyebut key:

- **Dot notation** `buku.judul` : nama key ditulis langsung di kode.
- **Bracket notation** `buku["judul"]` : nama key berupa **string**, jadi bisa berasal dari variabel.

```
const kolom = "tahun";
console.log(buku[kolom]);   // 2005   (sama dengan buku["tahun"])
console.log(buku.kolom);    // undefined (mencari key yang bernama "kolom")
```

Listing 14: key dari variabel

Bracket notation wajib dipakai ketika nama key baru diketahui saat program berjalan, misalnya dari data. Mirip memilih kolom secara dinamis di SQL.

2.4.3. Mengubah, menambah, dan menghapus key

Object yang disimpan di `const` tetap bisa diubah isinya. `const` hanya melarang variabel menunjuk ke object lain.

```
const akun = { pemilik: "Ani", saldo: 100000 };
akun.saldo = akun.saldo + 50000;   // ubah:   saldo 150000
akun.aktif = true;                 // tambah: key baru
delete akun.aktif;                 // hapus key
console.log(akun);                 // { pemilik: "Ani", saldo: 150000 }
// akun = {};                      // TypeError: Assignment to constant variable.
```

Listing 15: mengubah isi object

2.4.4. Object.keys, Object.values, Object.entries

Untuk apa: mengubah object menjadi array, supaya bisa diolah dengan method Modul 3.

```
const stok = { apel: 5, jeruk: 0, mangga: 12 };
console.log(Object.keys(stok));      // ["apel", "jeruk", "mangga"]
console.log(Object.values(stok));    // [5, 0, 12]
console.log(Object.entries(stok));   // [["apel", 5], ["jeruk", 0], ["mangga", 12]]

const habis = Object.entries(stok)
  .filter(([buah, jumlah]) => jumlah === 0)   // destructuring, lihat bagian berikut
  .map(([buah]) => buah);
console.log(habis);                  // ["jeruk"]
```

Listing 16: object menjadi array

Object juga bisa diulang dengan `for...in`, yang memberi **key** satu per satu: `for (const buah in stok) { console.log(buah, stok[buah]); }`. Ingat Modul 3: `for...in` dibuat untuk object. Untuk array, pakai `for...of`.

> **⚠ Perhatian**
>
> Object tidak punya `map`, `filter`, atau `length`. `stok.map(...)` melempar `TypeError: stok.map is not a function`, dan `stok.length` bernilai `undefined`. Ubah dulu dengan `Object.keys`, `Object.values`, atau `Object.entries`.

2.5. Pola: Object sebagai Penghitung

Untuk apa: menghitung berapa kali setiap nilai muncul. Contohnya jumlah transaksi per jenis atau jumlah buku per genre. Di SQL: `SELECT jenis, COUNT(*) ... GROUP BY jenis`.

Modul 3 mengerjakan GROUP BY dengan dua langkah (daftar unik, lalu `filter` per kelompok). Untuk sekadar **menghitung**, ada cara yang lebih pendek: satu object yang key-nya nama kelompok dan value-nya hitungan. Daftar kelompok tidak perlu dikumpulkan dulu. Key baru otomatis tercipta saat pertama kali muncul.

```
const transaksi = [
  { jenis: "setor", nominal: 50000 },
  { jenis: "tarik", nominal: 20000 },
  { jenis: "setor", nominal: 10000 },
  { jenis: "transfer", nominal: 75000 },
];

const hitung = {};
for (const t of transaksi) {
  if (hitung[t.jenis] === undefined) {   // jenis ini baru pertama kali muncul
    hitung[t.jenis] = 0;
  }
  hitung[t.jenis] = hitung[t.jenis] + 1;
}
console.log(hitung);   // { setor: 2, tarik: 1, transfer: 1 }
```

Listing 17: menghitung transaksi per jenis

Jalannya per putaran:

Putaran	`t.jenis`	`hitung[t.jenis]` sebelum	`hitung` sesudah
1	`"setor"`	`undefined`, jadi diisi 0	`{ setor: 1 }`
2	`"tarik"`	`undefined`, jadi diisi 0	`{ setor: 1, tarik: 1 }`
3	`"setor"`	`1`	`{ setor: 2, tarik: 1 }`
4	`"transfer"`	`undefined`, jadi diisi 0	`{ setor: 2, tarik: 1, transfer: 1 }`

Hal penting:

- Key-nya berasal dari data (`t.jenis`), jadi **wajib** bracket notation. `hitung.t.jenis` mencari key `t`, lalu gagal.
- Cek `undefined` diperlukan karena `undefined + 1` menghasilkan `NaN`. Tanpa cek, semua hitungan menjadi `NaN`.
- Urutan key mengikuti kemunculan pertama di data (untuk key berupa teks biasa).
- Untuk **menjumlahkan** (`SUM`), ganti `+ 1` dengan `+ t.nominal`.

2.6. Destructuring: Membongkar Object dan Array

Untuk apa: mengambil beberapa properti sekaligus ke variabel, tanpa menulis `buku.judul`, `buku.tahun`, dan seterusnya berulang kali.

2.6.1. Destructuring object

Cara kerja: di kiri tanda `=`, tulis kurung kurawal berisi nama key. JavaScript membuat variabel dengan nama yang sama dan mengisinya dari object di kanan.

```
const buku = { judul: "Laskar Pelangi", tahun: 2005 };

const { judul, tahun } = buku;        // sama dengan dua baris di bawah
// const judul = buku.judul;
// const tahun = buku.tahun;

const { judul: namaBuku } = buku;     // ganti nama variabel: namaBuku = buku.judul
const { harga = 0 } = buku;           // nilai default: key tidak ada, jadi harga = 0
console.log(judul, tahun, namaBuku, harga);   // Laskar Pelangi 2005 Laskar Pelangi 0
```

Listing 18: destructuring object

> **⚠ Perhatian**
>
> Nilai default hanya dipakai jika nilainya `undefined` (key tidak ada, atau isinya `undefined`). Jika isinya `null`, `0`, atau `""`, default **tidak** dipakai. `const { harga = 0 } = { harga: null }` menghasilkan `harga` bernilai `null`.

2.6.2. Destructuring di parameter fungsi

Destructuring paling sering dipakai langsung di parameter. Fungsi menerima satu object, tetapi langsung membongkarnya.

```
function labelBuku({ judul, tahun, tag = [] }) {
  return `${judul} (${tahun}) [${tag.join(", ")}]`;
}
console.log(labelBuku({ judul: "Bumi", tahun: 2014, tag: ["fantasi"] }));
// "Bumi (2014) [fantasi]"
console.log(labelBuku({ judul: "Pulang", tahun: 2015 }));
// "Pulang (2015) []"   (tag tidak ada, default [])
```

Listing 19: destructuring di parameter

Bentuk `labelBuku({ judul, tahun })` bisa langsung diberikan ke `map`: `daftarBuku.map(labelBuku)`. `map` memanggil `labelBuku` dengan satu object buku per putaran, lalu parameter membongkarnya.

Kesalahan umum: memanggil fungsi seperti ini tanpa argumen. `labelBuku()` melempar `TypeError: Cannot destructure property 'judul' of 'undefined' as it is undefined.`

2.6.3. Destructuring array

Untuk array, pakai kurung siku. Variabel diisi **berdasarkan posisi**, bukan nama.

```
const [tahun, bulan, hari] = "2026-11-04".split("-");
console.log(tahun, bulan, hari);   // "2026" "11" "04" (masih string)

const [pertama, , ketiga = "z"] = ["a", "b"];   // lewati posisi 2, default posisi 3
console.log(pertama, ketiga);      // "a" "z"
```

Listing 20: destructuring array

Pola `const [a, b, c] = teks.split(...)` sangat berguna untuk memecah satu baris data menjadi kolom-kolomnya.

2.7. Spread: Menyalin dan Menggabungkan

Untuk apa: membuat array atau object **baru** dari yang sudah ada, tanpa mengubah aslinya.

Cara kerja: tiga titik `...` "menumpahkan" isi array ke dalam array baru, atau isi object ke dalam object baru. Jika ada key yang sama, yang ditulis **belakangan** menang.

```
const a = [1, 2];
const b = [...a, 3];                // [1, 2, 3]
const gabung = [...a, ...b];        // [1, 2, 1, 2, 3]

const akun = { pemilik: "Ani", saldo: 100000 };
const salinan = { ...akun };                    // object baru, isi sama
const update = { ...akun, saldo: 150000 };      // saldo ditimpa
console.log(akun.saldo, update.saldo);          // 100000 150000
console.log(salinan === akun);                  // false (object berbeda)
```

Listing 21: spread pada array dan object

`{ ...akun, saldo: 150000 }` adalah cara standar "mengubah" object tanpa menyentuh aslinya. Mirip `UPDATE` yang menghasilkan baris baru, sementara baris lama tetap ada.

2.8. Referensi, Salinan Dangkal, dan Salinan Dalam

2.8.1. Variabel menyimpan alamat object

Number, string, dan boolean disimpan **sebagai nilainya**. Object dan array disimpan **sebagai alamat** (*referensi*) ke satu object di memori. Menyalin variabel berarti menyalin alamatnya, bukan object-nya.

```
const x = { n: 1 };
const y = x;          // y menyimpan alamat yang sama dengan x
y.n = 99;
console.log(x.n);     // 99 (object-nya cuma satu)
console.log(x === y); // true

console.log({ n: 1 } === { n: 1 });   // false: dua object berbeda, isi sama
```

Listing 22: dua variabel, satu object

`===` pada object membandingkan **alamat**, bukan isi. Analoginya di database: dua variabel yang memegang primary key yang sama menunjuk ke baris yang sama. Mengubah lewat salah satunya mengubah baris itu.

2.8.2. Spread hanya menyalin satu tingkat

`{ ...obj }` membuat object baru, tetapi isinya disalin apa adanya. Jika sebuah key berisi object lain (object bersarang), yang disalin adalah **alamat** object dalam itu. Ini disebut *salinan dangkal* (*shallow copy*).

```
const asli = { nama: "Ani", alamat: { kota: "Semarang" } };
const salinan = { ...asli };

salinan.nama = "Budi";              // tingkat 1: aman, asli tidak berubah
salinan.alamat.kota = "Solo";       // tingkat 2: object alamat dipakai bersama
console.log(asli.nama);             // "Ani"
console.log(asli.alamat.kota);      // "Solo"  (ikut berubah)
console.log(salinan.alamat === asli.alamat);   // true
```

Listing 23: salinan dangkal

Bagian	`asli`	`salinan`
object luar	object A	object B (baru, hasil spread)
`nama`	`"Ani"`	`"Budi"` (disalin nilainya, lalu diganti)
`alamat`	alamat ke object K	alamat ke object K yang **sama**

2.8.3. Cara membuat salinan dalam

Ada dua cara yang umum:

1. **Spread di setiap tingkat yang akan diubah.** Salin object luar, lalu salin juga object dalam yang mau diganti. Cara ini paling sering dipakai karena jelas bagian mana yang berubah.
2. **`structuredClone(obj)`.** Menyalin semua tingkat sekaligus (*deep copy*). Tersedia di Node 17 ke atas dan browser modern.

```
const asli = { nama: "Ani", alamat: { kota: "Semarang" } };   // mulai lagi dari data awal

const baru1 = { ...asli, alamat: { ...asli.alamat, kota: "Solo" } };
console.log(asli.alamat.kota, baru1.alamat.kota);   // "Semarang" "Solo"
console.log(baru1.alamat === asli.alamat);          // false

const baru2 = structuredClone(asli);
baru2.alamat.kota = "Solo";
console.log(asli.alamat.kota);                      // "Semarang"
```

Listing 24: dua cara salinan dalam

Baca `{ ...asli, alamat: { ...asli.alamat, kota: "Solo" } }` dari luar ke dalam. Buat object baru berisi semua key `asli`. Lalu key `alamat` ditimpa dengan object baru, yang berisi semua key `asli.alamat`, dengan `kota` ditimpa `"Solo"`.

✎ Catatan

Modul 3 punya masalah serupa: `sort` mengubah array asli. Di sana solusinya `[...arr].sort(...)`. Itu juga salinan dangkal. Aman karena `sort` hanya memindahkan posisi elemen, tidak mengubah isi object di dalamnya.

2.9. JSON: Object dalam Bentuk Teks

Untuk apa: menyimpan atau mengirim data sebagai teks, misalnya ke file atau lewat internet (REST API di modul nanti). JSON (*JavaScript Object Notation*) adalah format teks yang mirip object literal.

Cara kerja:

- `JSON.stringify(nilai)` mengubah object atau array menjadi **string** JSON.
- `JSON.parse(teks)` mengubah string JSON menjadi object atau array **baru**.

```
const mhs = { nama: "Ani", nilai: [80, 90], aktif: true };
const teks = JSON.stringify(mhs);
console.log(teks);           // {"nama":"Ani","nilai":[80,90],"aktif":true}
console.log(typeof teks);    // "string"

const kembali = JSON.parse(teks);
console.log(kembali.nilai[1]);   // 90
console.log(kembali === mhs);    // false (object baru)

console.log(JSON.stringify({ a: 1, b: [1, 2] }, null, 2));   // dengan indentasi 2 spasi
```

Listing 25: stringify dan parse

Aturan JSON lebih ketat daripada object literal:

- Key **wajib** diberi kutip dua: `{"nama":"Ani"}`. Kutip satu tidak boleh.
- Nilainya hanya boleh string, number, boolean, `null`, array, dan object. Fungsi dan `undefined` dibuang saat `stringify`.

```
console.log(JSON.stringify({ nama: "Ani", umur: undefined, hobi: ["baca"] }));
// {"nama":"Ani","hobi":["baca"]}   (umur hilang)
```

Listing 26: yang hilang saat stringify

⚠ Perhatian

`JSON.parse` melempar `SyntaxError` jika teksnya bukan JSON yang sah:

- `JSON.parse("{nama: 'Ani'}")`: `SyntaxError: Expected property name or '}' in JSON at position 1 (line 1 column 2)`. Key tanpa kutip dua.
- `JSON.parse("")`: `SyntaxError: Unexpected end of JSON input`. Teks kosong.

Jangan memanggil `JSON.parse` pada sesuatu yang sudah berupa object.

2.10. Membaca Pesan Error

Pesan error	Artinya	Penyebab umum
`x.padStart is not a function` (atau `toUpperCase`, `split`, dan lainnya)	`x` bukan string, jadi tidak punya method string.	`x` berupa number. Ubah dengan `String(x)`.
`x.join is not a function`	`x` bukan array.	`x` berupa string. Mungkin lupa `split`.
`x.map is not a function`	`x` bukan array.	`x` berupa object. Pakai `Object.entries(x)` dulu.

`Cannot read properties of undefined (reading 'toUpperCase')`	Kamu menulis `x.toUpperCase()`, dan `x` bernilai `undefined`.	`kata[0]` pada string kosong. Key object salah tulis.
`Cannot assign to read only property '0' of string '...'`	Menulis ke index string.	String tidak bisa diubah. Susun string baru.
`Cannot destructure property 'x' of 'undefined'`	Destructuring dari `undefined`.	Fungsi dipanggil tanpa argumen.
`SyntaxError ... in JSON at position N`	Teks bukan JSON yang sah.	Key tanpa kutip dua, kutip satu, koma di akhir.

Cara melacaknya sama dengan Modul 3. Lihat nilai di sebelah kiri titik, lalu `console.log` nilai itu beserta `typeof`-nya tepat sebelum baris error.

Catatan

Saat live coding, pertanyaan yang sering muncul adalah “apa beda salinan dangkal dan dalam?” Jawaban yang baik menyebut tiga hal. Pertama, variabel object menyimpan alamat. Kedua, spread hanya membuat object baru di tingkat pertama, sehingga object bersarang tetap dipakai bersama. Ketiga, solusinya spread di setiap tingkat atau `structuredClone`.

Coba Sendiri

Kerjakan di fungsi `cobaSendiri()` pada file soal `Jawaban/soal-js4.js` (sudah tersedia di starter). Sebelum menjalankan, **tulis tebakanmu** sebagai komentar di ujung setiap baris `console.log`. Jalankan bagian ini saja dengan `node Jawaban/soal-js4.js coba`, lalu bandingkan.

```
const kata = "Halo";
console.log(kata.length);
console.log(kata[0], kata[10]);
console.log(kata.toUpperCase(), kata);
console.log("a,b,,c".split(","));
console.log("7".padStart(3, "0"));

const a = { n: 1, dalam: { m: 2 } };
const b = { ...a };
b.n = 9;
b.dalam.m = 9;
console.log(a.n, a.dalam.m);
console.log({ x: 1 } === { x: 1 });
console.log(JSON.stringify({ nama: "Ani", umur: undefined, hobi: ["baca"] }));
```

1. Tebakan mana yang salah? Jelaskan penyebabnya.
2. Mengapa `a.n` tetap `1`, tetapi `a.dalam.m` menjadi `9`?
3. Mengapa `{ x: 1 } === { x: 1 }` bernilai `false`?
4. Tambahkan satu baris yang mencetak semua key object `a` dalam satu string dipisah koma: `n, dalam`. Petunjuk: `Object.keys` lalu `join`.

Jawab pertanyaan di atas di bawah baris `// JAWABAN Coba Sendiri N` pada file soal.